ApplyWizz DicePilot

Document 06 - Implementation Plan

The exact build sequence for Claude Code - one milestone at a time

Product	ApplyWizz DicePilot
Version	V1.0 - Pilot Specification
Scope	V1 only: one internal test candidate, up to 20 Dice C2C + Easy Apply jobs, sequential processing
Date	20 August 2026

1. Implementation Strategy

Build the smallest reliable vertical slice first. Do not ask Claude Code to build the final product in one prompt. Each phase below has a defined completion gate. No later phase should begin until the previous phase has tests/evidence and existing Indeed functionality remains intact.

Core rule for the coding agent
Read the repository before editing. Preserve existing Indeed functionality. Add Dice in isolated modules. Never invent candidate facts. Never mark SUBMITTED without positive verification. Do not implement multi-candidate, Telegram, recruiter email, or resume AI in V1.

2. Phase 0 - Repository Baseline and Safety

Task	Done criteria
Create feature branch/worktree	Work is isolated from main; existing files preserved.
Run existing app/tests/smoke checks	Record baseline behavior and known pre-existing failures.
Inventory current Supabase usage	Identify hardcoded secrets and table dependencies.
Create docs/ source-of-truth directory	Place these six specs and coding rules where coding agent can reference them.
Define V1 feature flag/config	DicePilot code can be disabled without affecting Indeed flow.

3. Phase 1 - Shared Configuration Cleanup

Task	Done criteria
Create shared Supabase client helper	New code reads SUPABASE_URL and server key from environment.
Remove new hardcoded secrets	No new credentials committed; existing risky hardcoding flagged/fixed safely.
Add structured logging	Dice modules use consistent log format and application IDs.
Add configuration validation	Worker fails fast with clear missing-variable error before browser launch.

4. Phase 2 - Database Migrations

1. Add dice_jobs table and unique dice_job_id.
2. Add applications table with UNIQUE(candidate_id, dice_job_id).
3. Add application_events.
4. Add interventions.
5. Add browser_profiles.
6. Add required indexes and status constraints.
7. Add repository layer with create/get/claim/update/event methods.
8. Write migration tests or a schema verification script.

Phase 2 gate

Can create a test Dice job, queue it once for the pilot candidate, reject the duplicate, append events, and retrieve the ordered queue without launching a browser.

5. Phase 3 - Dice Discovery Skeleton

Task	Implementation
Create dice package	Add scraper.py, job_parser.py, c2c_classifier.py, easy_apply_detector.py with typed interfaces.
Search input	Start with one controlled role/search configuration and US scope.
Employment filter	Use Dice Contract + Third Party search/filter semantics where available.
Job identity	Normalize Dice job ID and canonical URL.
Full description	Persist full job description for C2C analysis.
Dedup	Use normalized Dice job ID, not display title, as primary identity.

6. Phase 4 - C2C Classifier

9. Implement deterministic phrase/evidence rules before adding any LLM classification.
10. Positive evidence examples include C2C, Corp-to-Corp, Corp to Corp, corporation-to-corporation, third-party candidates accepted where unambiguous.
11. Negative evidence examples include no C2C, W2 only, no third parties, direct hire only.
12. Return class + reason/evidence.
13. Write unit tests for positive, negative, conflicting, and no-evidence descriptions.
14. Do not let CONTRACT alone become CONFIRMED C2C.

Phase 4 gate
Given a fixture set of descriptions, classifier returns stable CONFIRMED / LIKELY / NOT_C2C / UNKNOWN results with tests.

7. Phase 5 - Easy Apply Detector

15. Implement search-level Easy Apply filter if current Dice supports it.
16. Implement job-detail positive detector using current Dice controls; use the Playwright patterns from researched Dice repos only as starting references.
17. Differentiate Dice Easy Apply from external redirects.
18. Persist detector evidence/version.
19. Write detector fixtures/mocks and a live controlled dry-run path.
20. Never submit during this phase.

Phase 5 gate
System can produce a list of Dice jobs where C2C status and Easy Apply status are both known, without applying to anything.

8. Phase 6 - 20-Job Queue Creation

21. Choose the one authorized V1 candidate ID.
22. Fetch candidate excluded-company list.
23. Select up to 20 eligible Dice jobs after exclusion and duplicate checks.
24. Create application rows as QUEUED.
25. Display the queue in the internal UI or a simple API/CLI report.
26. Freeze the test set for the first controlled execution run so debugging is reproducible.

9. Phase 7 - Candidate Adapter

Task	Done criteria
HTTP client	Can fetch candidate by ID from existing ApplyWizz route with proper timeout/error handling.
Typed model	Normalized candidate fields are typed and validated.
Precedence rules	Conflicting/null work-auth/contact fields follow explicit documented rules.
Resume resolver	Can obtain/download the candidate resume to a safe temporary path.
Privacy logging	No full raw candidate payload appears in normal logs.
Unit tests	Fixtures cover null/conflicting fields without using real PII.

10. Phase 8 - Persistent Dice Browser

27. Implement browser profile manager using Playwright persistent context or equivalent isolated profile directory.
28. Create one profile mapping for the pilot candidate.
29. Support headful V1 debugging.
30. Verify login/session state on startup and before application execution.
31. If login/security action is required, record NEEDS_INPUT rather than attempting bypass.
32. Add candidate execution lock so only one application can use the profile at a time.
33. Capture trace/screenshot on session errors.

Phase 8 gate
Close/restart the worker and prove the same authorized candidate profile can retain/recover the Dice session as intended, or cleanly request re-authentication.

11. Phase 9 - Easy Apply Vertical Slice for One Job

34. Select exactly one known eligible test job.
35. OPEN_JOB and verify correct Dice job.
36. CHECK_LOGGED_IN.
37. CHECK_ELIGIBILITY and already-applied condition.
38. CLICK_EASY_APPLY.
39. DISCOVER_FORM.
40. HANDLE_RESUME using existing verified resume.
41. Do not submit yet until the next phase proves question mapping.
42. Persist state events throughout.

12. Phase 10 - Form Question Mapping

43. Inventory actual question/control types observed on controlled Dice Easy Apply jobs.
44. Implement direct mappings for known fields only.
45. Support text, radio/select, checkbox, and numeric fields only as actually encountered and tested.
46. Use normalized candidate facts; do not call an LLM to invent answers.
47. Unknown required question -> create intervention + set NEEDS_INPUT.
48. Write unit tests for question normalization and answer mapping.
49. Resume a controlled intervention only from a defined state.

13. Phase 11 - Submission and Verification

50. Implement final review/submit state.
51. Before click, set status SUBMITTING and write event.
52. Prevent duplicate final-click retries when result is uncertain.
53. Implement positive success verification from current Dice UI/state.
54. Only after positive verification set SUBMITTED + submitted_at + evidence.
55. If success cannot be confirmed, do not assume success; use failure/uncertain policy.
56. Capture final evidence reference.

Phase 11 gate
One controlled eligible Dice application can go from QUEUED to a positively verified SUBMITTED state, or to a safe NEEDS_INPUT/FAILED outcome, with a complete event timeline.

14. Phase 12 - Sequential Worker Loop

57. Atomically claim next QUEUED application for the candidate.
58. Run full state machine.
59. On terminal outcome release application ownership.
60. If NEEDS_INPUT blocks the current job, define whether V1 pauses the pilot or continues to the next job; default recommendation: continue to next independent job while keeping only one browser flow active at a time and only if browser state is clean.
61. Stop when no queue items remain or operator pauses.
62. Do not run multiple applications concurrently for the same candidate.

15. Phase 13 - V1 Monitoring UI

Component	Done criteria
Dashboard summary	Shows processed/submitted/failed/needs-input/remaining.
Browser status	Shows candidate Dice session/worker status.
Current application	Shows current job and state step.
Queue table	All pilot jobs and latest statuses.
Application detail	Timeline and sanitized failure/intervention context.
Needs Input	Operator can inspect exact blocker and provide allowed response through controlled workflow.

16. Phase 14 - 20-Job Pilot Execution

63. Freeze code version and selector/config version for the run.

64. Prepare up to 20 verified C2C + Easy Apply jobs for the test candidate.
65. Confirm candidate API and browser session.
66. Start worker and observe sequential execution.
67. Do not manually edit database statuses to make the dashboard look successful.
68. Record every outcome and failure class.
69. After run, calculate submitted / failed / needs input / already applied or ineligible changes.
70. Review screenshots/traces for failures and identify top selector/question gaps.
71. Decide whether another V1 reliability iteration is needed before any V2 multi-candidate work.

17. Testing Matrix

Test area	Minimum V1 tests
C2C classifier	Positive, negative, likely, unknown, conflicting language.
Candidate adapter	Nulls, conflicting sponsorship fields, excluded companies, resume URL, invalid phone.
Repository/idempotency	Duplicate candidate/job rejected; atomic queue order; legal status transitions.
Question mapper	Known yes/no, numeric, dropdown, unknown required field -> NEEDS_INPUT.
Browser session	Authenticated, expired, login/security challenge.
Easy Apply	Present, absent, external redirect, already applied.
Submission	Verified success, click failure, uncertain result.
Worker	Sequential processing and candidate lock.

18. Claude Code Working Rules

72. Before each phase, read the relevant existing files and these six documents.
73. Propose the exact files to change before editing.
74. Make small commits per phase or milestone.
75. Run tests after every meaningful change.
76. Do not rewrite unrelated existing scraper code for style preferences.
77. Do not introduce a second database or duplicate candidate-profile service.
78. Do not add generic AI agents where deterministic code is sufficient.
79. Do not implement security-control bypass behavior.
80. Do not claim a phase is complete without evidence from tests/run output.
81. Stop at V1 boundaries even if future features seem easy.

19. Final V1 Done Definition

V1 is done when
One authorized test candidate can be assigned up to 20 verified Dice C2C + Easy Apply jobs; one persistent Dice browser processes them sequentially; known fields come from verified ApplyWizz candidate data; unknown/security items stop safely; every application has an auditable state/event history; and SUBMITTED is used only after positive Dice success verification.

20. Do Not Start V2 Until

- The 20-job run has been reviewed.
- Top failure modes are understood.
- Duplicate and false-success protections are validated.
- Candidate data mapping is stable.
- Persistent browser/session behavior is reliable enough for another user.
- The team explicitly approves moving to multi-candidate architecture.